Experiment No: 07

Experiment Title

Implementation of Backtracking Search to Solve the 8-Queens Constraint Satisfaction Problem.

Aim

To implement the Backtracking Search algorithm to solve the 8-Queens Constraint Satisfaction Problem and place eight queens on a chessboard such that no two queens attack each other.

Objective

- To understand Constraint Satisfaction Problems (CSPs).
- To study the Backtracking Search algorithm.
- To solve the 8-Queens Problem using backtracking.
- To understand constraint checking and state-space search.
- To find a valid arrangement of eight queens on a chessboard.

Theory

The **8-Queens Problem** is one of the classical Constraint Satisfaction Problems (CSP) in Artificial Intelligence. The objective is to place eight queens on an 8×8 chessboard such that no two queens attack each other.

According to the rules of chess, a queen can attack another queen if they are placed:

- In the same row.
- In the same column.
- On the same diagonal.

Therefore, the solution must satisfy the following constraints:

1. No two queens should be in the same row.
2. No two queens should be in the same column.
3. No two queens should lie on the same diagonal.

Constraint Satisfaction Problem (CSP)

A CSP consists of:

- **Variables:** Positions of queens.
- **Domain:** Possible rows where queens can be placed.
- **Constraints:** Queens must not attack each other.

Backtracking Search

Backtracking is a systematic search technique used to solve CSPs. It incrementally builds a solution and abandons a partial solution as soon as it determines that the solution cannot be completed successfully.

The algorithm works as follows:

1. Place a queen in the first column.
2. Check whether the position is safe.
3. If safe, move to the next column.
4. If no safe position exists, backtrack to the previous column.
5. Continue until all eight queens are placed.

Advantages of Backtracking

- Efficient for constraint satisfaction problems.
- Avoids exploring invalid solutions.
- Easy to implement.

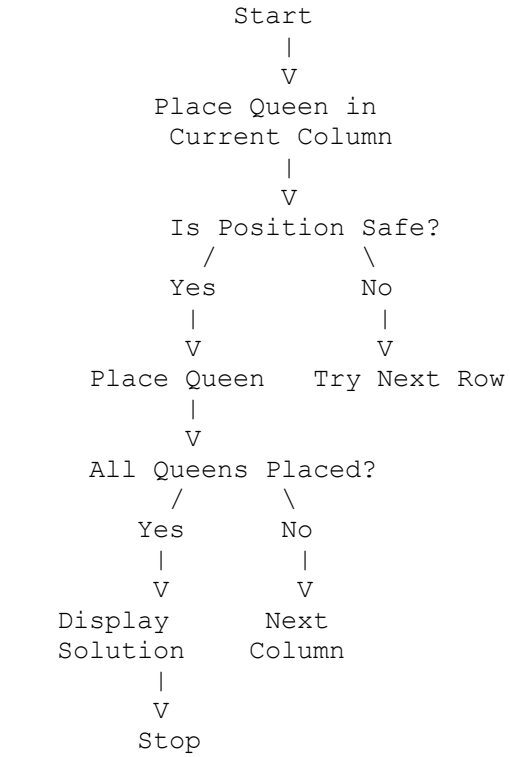
Limitations

- Can be slow for large problems.
- Requires recursive calls.
- Worst-case time complexity is high.

Algorithm

1. Start.
2. Place a queen in the first column.
3. Check whether the current position is safe.
4. If safe, place the queen and move to the next column.
5. If not safe, try the next row.
6. If all rows are exhausted, backtrack.
7. Repeat until all eight queens are placed.
8. Display the solution.
9. Stop.

Flowchart



Python Program

```
# 8 Queens using Backtracking

N = 8

def is_safe(board, row, col):

    for i in range(col):
        if board[row][i] == 1:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
```

```
    return True

def solve(board, col):

    if col >= N:
        return True

    for row in range(N):

        if is_safe(board, row, col):

            board[row][col] = 1

            if solve(board, col + 1):
                return True

            board[row][col] = 0

    return False

board = [[0]*N for _ in range(N)]

if solve(board, 0):

    for row in board:
        print(row)

else:
    print("No Solution")
```

Sample Output

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
```

Observation

Queen Number	Position (Row, Column)
Q1	(0,0)
Q2	(6,1)
Q3	(4,2)
Q4	(7,3)

Queen Number	Position (Row, Column)
Q5	(1,4)
Q6	(3,5)
Q7	(5,6)
Q8	(2,7)

Applications

1. Constraint Satisfaction Problems (CSP).
2. Scheduling and Timetabling.
3. Resource Allocation.
4. Puzzle Solving.
5. Artificial Intelligence Planning.
6. Job Assignment Problems.
7. Circuit Design.
8. Robot Motion Planning.

Result

The 8-Queens Constraint Satisfaction Problem was successfully solved using the Backtracking Search algorithm. Eight queens were placed on the chessboard such that no two queens attacked each other.

Viva Questions

1. What is the 8-Queens Problem?
2. What is a Constraint Satisfaction Problem?
3. What is Backtracking?
4. Why is the 8-Queens Problem considered a CSP?
5. What constraints are used in the 8-Queens Problem?
6. How does Backtracking work?
7. What is a safe position?
8. What is recursion?
9. What are the applications of Backtracking?
10. What is the advantage of Backtracking over brute force?
11. What is the time complexity of the 8-Queens Problem?
12. What is a state space tree?

13. Can multiple solutions exist for the 8-Queens Problem?
14. What happens when no safe position is found?
15. What is the role of constraint checking?

Conclusion

The Backtracking Search algorithm successfully solved the 8-Queens Constraint Satisfaction Problem by systematically placing queens and backtracking whenever a constraint violation occurred. This experiment demonstrated the effectiveness of backtracking in solving CSPs and optimization problems in Artificial Intelligence.